Introducción a K-Means

Aprendizaje No Supervisado

UDEM Jorge Alberto Ramírez García, M.Eng.

¿Qué es K-Means?

Agrupa puntos parecidos alrededor de **k centros** (centroides), sin etiquetas.

Tú decides

- El número de grupos **k**
- Qué variables entran
- Cómo se escalan

El algoritmo decide

- Qué punto va en qué grupo
- Dónde queda cada centroide

Se usa para

- Segmentar clientes
- Agrupar productos o tiendas
- Resumir datos en prototipos

No hay respuesta correcta contra la cual comparar: el valor está en que los grupos tengan **sentido** y lleven a una **acción**.

El algoritmo de Lloyd

1. **Inicializar**
   Elegir k puntos como centroides iniciales (k-means++ los elige separados)

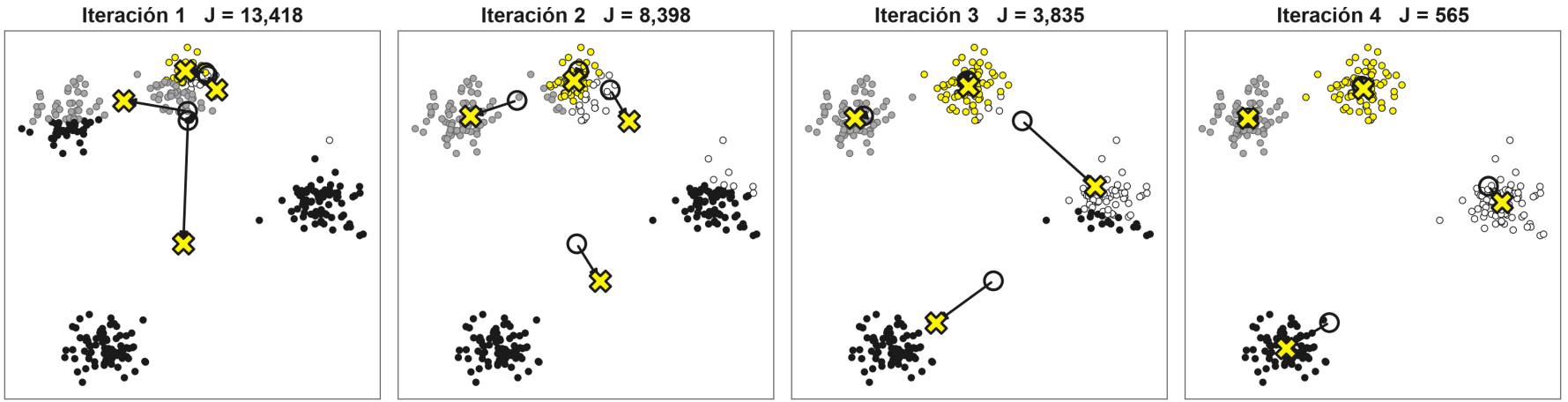
2. **Asignar**
   Cada punto se va con el centroide más cercano (distancia euclidiana)

3. **Recalcular**
   Cada centroide se mueve al promedio de los puntos que le tocaron

4. **Repetir 2 y 3**
   Hasta que ningún punto cambie de cluster (los centroides ya no se mueven)

Cada paso **baja o mantiene** la inercia, así que el algoritmo **siempre converge**… aunque puede quedarse en un óptimo local.

Lloyd en acción: asignar → recalcular



○ centroide antes → ✕ centroide después de recalcular. Color = cluster asignado.

Arranque malo a propósito (los 4 centroides en la misma nube): en 4 iteraciones cada nube tiene su centro.

¿Qué optimiza? La inercia

$$J = \Sigma_i \|x_i - \mu_c(i)\|^2$$

Se lee así

- Suma de distancias² de cada punto a **su** centroide
- Pequeña = grupos apretados

Por qué baja

- **Asignar**: cada punto elige su centro más cercano
- **Recalcular**: el promedio minimiza la suma de distancias²

Cuidado

- **Siempre** baja al subir k (con k = n vale 0)
- Sirve para el codo, no para comparar k directamente

Inicialización: k-means++ y n_init

El problema: óptimos locales

- Si dos centroides arrancan en la misma nube, Lloyd converge a una solución peor
- Ningún paso puede arreglarlo después

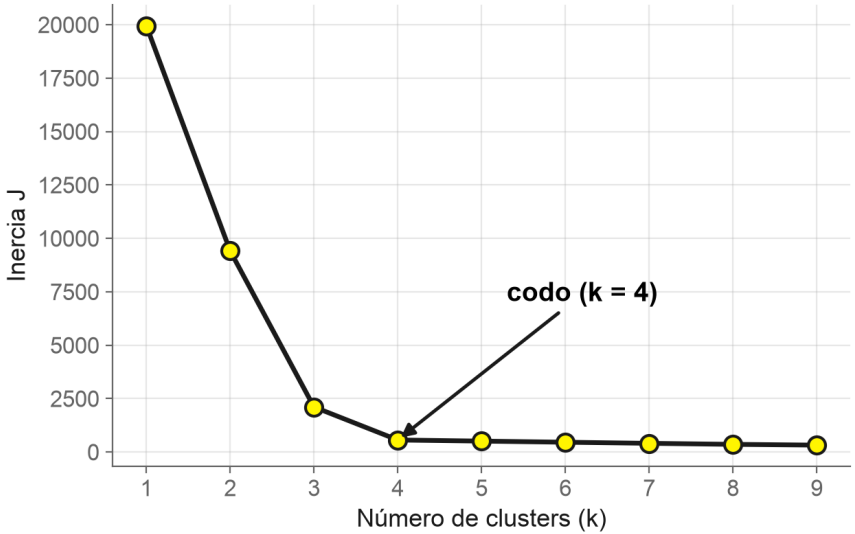
Los remedios

- **k-means++** (default): cada centroide nuevo se elige lejos de los anteriores
- **n_init=10**: 10 arranques, se queda con el de menor inercia

```
KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
```

Desde scikit-learn 1.4, n_init='auto' hace un solo intento con k-means++: escríbelo explícito.

¿Cómo elegir k?



Señal	Buscamos
Inercia (codo)	Donde deja de bajar mucho
Silhouette	Alto (−1 a 1)
Gráfica de silueta	Clusters parejos, sin negativos

Usa **codo, silhouette y el sentido de negocio**. Cuando no coinciden, dilo y prefiere la solución más simple que se pueda explicar y accionar.

Escala antes de agrupar

Sin escalar

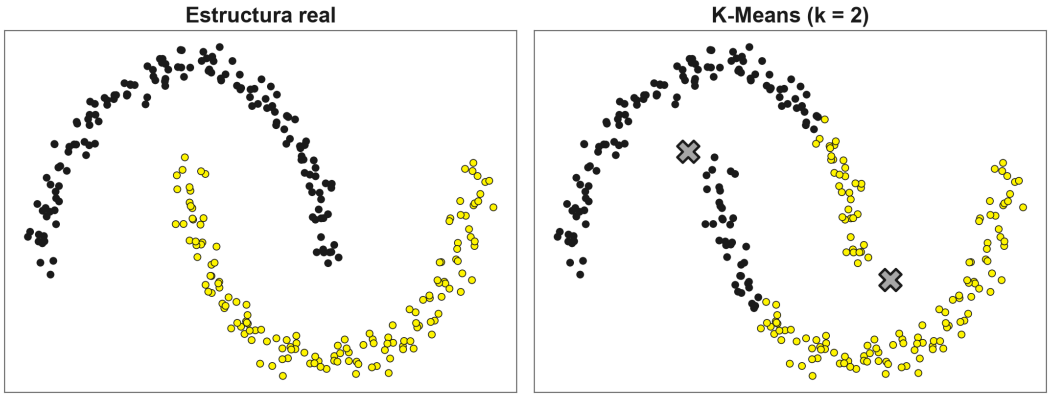
- Edad: 18 a 70 (años)
- Ingreso: 5,000 a 150,000 (pesos)
- La distancia es casi sólo **ingreso**: la edad no cuenta
- Cambiar de unidades cambia los clusters

Con StandardScaler

- Cada variable: media 0, desviación 1
- Todas pesan lo mismo
- El resultado no depende de las unidades

Ajusta con los datos escalados, pero **interpreta en unidades originales** (scaler.inverse_transform o promedios por cluster sobre el df original).

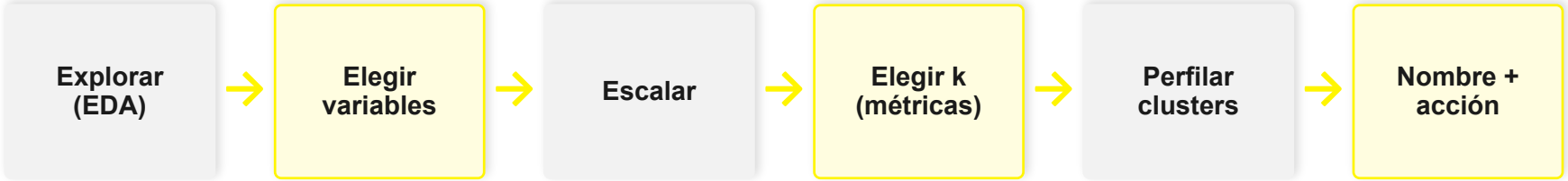
Cuándo K-Means falla



- Fronteras **rectas** entre centroides
- Prefiere grupos **redondos** y de tamaño parecido
- Falla con lunas, anillos y densidades distintas
- Todo punto pertenece a un cluster: **no hay ruido**
- Sensible a atípicos (arrastran el promedio)

Siguiente sesión: **DBSCAN** agrupa por zonas densas, sigue formas raras y marca el ruido.

Segmentación en negocio: el flujo



Perfilar

- Tamaño y promedio de cada variable por cluster
- En unidades que entienda el negocio
- El número del cluster no significa nada

Accionar

- Un **nombre** por segmento
- Una **campaña** o decisión por segmento
- Si no cambia ninguna decisión, sobra el segmento

Código mínimo

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans


X = StandardScaler().fit_transform(df[columnas])      # 1. escalar
tabla = tabla_metricas_k(X, range(2, 11))             # 2. codo y silhouette por k
kmeans = KMeans(n_clusters=k, n_init=10,
                random_state=42).fit(X)               # 3. ajustar
perfil_clusters(df, kmeans.labels_, columnas)         # 4. perfilar
```

kmeans.labels_ = cluster de cada fila · kmeans.cluster_centers_ = centroides (escalados) · kmeans.inertia_ = J

Errores comunes

Error	Qué hacer
No escalar	StandardScaler antes; perfiles en unidades originales
Un solo intento	n_init=10 y random_state=42 explícitos
Elegir k sólo con el codo	Codo + silhouette + negocio
Interpretar el número del cluster	Ponle nombre según su perfil
Meter dummies 0/1 sin pensar	Dominan la distancia: perfílalas después
Usarlo con formas raras o ruido	Grafica antes; prueba DBSCAN

Conclusiones

1. K-Means alterna asignar y recalcular hasta minimizar la inercia J
2. k-means++ y n_init=10 evitan quedarse en un óptimo local
3. Elige k con codo, silhouette y sentido de negocio
4. Escala antes de agrupar e interpreta en unidades originales
5. Grupos redondos sí; formas raras, densidades distintas y ruido: DBSCAN